Lab 20 · Self-paced

SAML Extension Grant

WSO2 API Manager 4.7.0 · Exchanging a SAML 2.0 assertion for an OAuth2 access token

Before You Begin

URLs and paths in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – API Manager HTTPS port: 9443 plus any port offset (Management Console, Developer Portal, token endpoint).
- **`<GW_HTTPS_PORT>`** – API Gateway HTTPS port: 8243 plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The server uses a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue. The -k option in the curl commands skips the same check.

Some values below contain `https://localhost:9443/oauth2/token`. Type them exactly as shown, even if your server uses another host or port: they are names that must match inside the SAML assertion, not addresses that are called.

Training Objective

Learn how an application that already has a SAML 2.0 assertion from a trusted identity provider (IdP) can exchange it for an OAuth2 access token (the SAML2 bearer grant) and call an API with that token.

Business Scenario

PizzaShack's enterprise applications use SAML single sign-on with a trusted IdP. They want to call APIs with an OAuth2 token without asking users to log in again, by exchanging the SAML assertion they already have.

High-Level Steps

- Register the trusted IdP in API Manager
- Enable the SAML2 grant on the application
- Create a signed SAML assertion (the lab uses a small tool in place of a real IdP)
- Exchange the assertion for an OAuth2 access token and call the API

Prerequisites

- An OAuth2-protected API (for example PizzaShack) and an application subscribed to it.
- A JDK (for `keytool` and `java`).
- Lab file: `saml2-assertion-creator.zip`.

How the lab works: in a real set-up the IdP (for example WSO2 Identity Server) issues the SAML assertion after single sign-on and signs it with its own key. Here the SAML2AssertionCreator tool plays the IdP: it signs the assertion with API Manager's own default key, so that key's certificate is registered as the IdP certificate.

CC BY 4.0

Part 1 – Register the Trusted IdP

1. Export the certificate that will sign the assertions. `keytool` is in your JDK's bin folder. This creates `idp.crt` in the current folder.

   macOS / Linux

   ```
   keytool -exportcert -rfc \
     -alias wso2carbon \
     -keystore <APIM_HOME>/repository/resources/security/wso2carbon.jks \
     -storepass wso2carbon \
     -file idp.crt
   ```

   Windows (Command Prompt)

   ```
   keytool -exportcert -rfc ^
     -alias wso2carbon ^
     -keystore <APIM_HOME>\repository\resources\security\wso2carbon.jks ^
     -storepass wso2carbon ^
     -file idp.crt
   ```

2. Log in to the Management Console as admin and click **Main** → **Identity Providers** → **Add**.

   Management Console URL

   ```
   https://<APIM_HOST>:<HTTPS_PORT>/carbon
   ```

3. Under **Basic Information**, enter:

Field	Value
Identity Provider Name	`TestIdP`
Identity Provider Public Certificate	Choose to upload a certificate and select idp.crt
Alias (Token Endpoint Alias)	`https://localhost:9443/oauth2/token`

   The alias is the token endpoint named as the audience in the assertion.

4. Expand **Federated Authenticators** → **SAML2 Web SSO Configuration**, tick **Enable SAML2 Web SSO** and enter:

Field	Value
Identity Provider Entity ID	`TestSP`
Service Provider Entity ID	`APIM`
SSO URL	`https://localhost:9444/samlsso`

   The Identity Provider Entity ID must equal the issuer in the assertion. The SSO URL is required by the form but not used by the token exchange. Click **Register**.

Part 2 – Enable the SAML2 Grant on the Application

5. In the Developer Portal, open your application and go to **Production Keys** → **OAuth2 Tokens**.

   Developer Portal URL

   ```
   https://<APIM_HOST>:<HTTPS_PORT>/devportal
   ```

CC BY 4.0

6. Under **Grant Types**, tick **SAML2** in addition to **Client Credentials**, then click **Generate Keys** (or **Update** if keys already exist).

   The SAML2 grant type is `urn:ietf:params:oauth:grant-type:saml2-bearer`. Copy the **Consumer Key** and **Consumer Secret**; you need them in step 9.

Part 3 – Create a Signed SAML Assertion

7. Extract `saml2-assertion-creator.zip` and open a terminal in the `SAML2AssertionCreator` folder.

8. Create an assertion for the user `admin`. The arguments, in order, are:

#	Argument	Value in this lab
1	IdP entity ID (issuer)	`TestSP`
2	Subject (user)	`admin`
3	Recipient	`https://localhost:9443/oauth2/token`
4	Audience	`https://localhost:9443/oauth2/token`
5	Keystore	`<APIM_HOME>/repository/resources/security/wso2carbon.jks`
6	Keystore password	`wso2carbon`
7	Key alias	`wso2carbon`
8	Key password	`wso2carbon`

macOS / Linux

```
java -jar SAML2AssertionCreator.jar \
  TestSP \
  admin \
  https://localhost:9443/oauth2/token \
  https://localhost:9443/oauth2/token \
  <APIM_HOME>/repository/resources/security/wso2carbon.jks \
  wso2carbon wso2carbon wso2carbon
```

Windows (Command Prompt)

```
java -jar SAML2AssertionCreator.jar ^
  TestSP ^
  admin ^
  https://localhost:9443/oauth2/token ^
  https://localhost:9443/oauth2/token ^
  <APIM_HOME>\repository\resources\security\wso2carbon.jks ^
  wso2carbon wso2carbon wso2carbon
```

The tool prints the assertion XML and then a line starting with:

```
base64-url Encoded Assertion String:
```

Copy the long value after it. The assertion is valid for 5 minutes, so do the next step straight away.

CC BY 4.0

Part 4 – Exchange the Assertion and Call the API

9. Exchange the assertion for an access token. Replace `<CONSUMER_KEY>`, `<CONSUMER_SECRET>` and `<ENCODED_ASSERTION>`. The `-u` option sends the key and secret as the Basic authorization header.

macOS / Linux

```
curl -k \
  -u <CONSUMER_KEY>:<CONSUMER_SECRET> \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=urn:ietf:params:oauth:grant-type:saml2-bearer" \
  -d "assertion=<ENCODED_ASSERTION>" \
  -d "scope=PRODUCTION" \
  https://<APIM_HOST>:<HTTPS_PORT>/oauth2/token
```

Windows (Command Prompt)

```
curl -k ^
  -u <CONSUMER_KEY>:<CONSUMER_SECRET> ^
  -H "Content-Type: application/x-www-form-urlencoded" ^
  -d "grant_type=urn:ietf:params:oauth:grant-type:saml2-bearer" ^
  -d "assertion=<ENCODED_ASSERTION>" ^
  -d "scope=PRODUCTION" ^
  https://<APIM_HOST>:<HTTPS_PORT>/oauth2/token
```

Expected response (shortened):

```
{
  "access_token": "<ACCESS_TOKEN>",
  "scope": "PRODUCTION",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

10. Call the API with that token:

macOS / Linux

```
curl -k -i \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

Windows (Command Prompt)

```
curl -k -i ^
  -H "Authorization: Bearer <ACCESS_TOKEN>" ^
  https://<APIM_HOST>:<GW_HTTPS_PORT>/pizzashack/1.0.0/menu
```

Expected: `HTTP/1.1 200 OK` and the pizza menu in JSON.

CC BY 4.0

Troubleshooting

Problem	What to check
Token request returns invalid_grant	Create a new assertion: it may have expired (5 minutes).
invalid_grant with a fresh assertion	The IdP Entity ID in TestIdP is TestSP, and the Alias equals the recipient/audience used in the assertion (https://localhost:9443/oauth2/token).
unauthorized_client	The SAML2 grant type is ticked on the application keys (step 6) and the key/secret are from the same application.
API call returns 401 or 403	The application is subscribed to the PizzaShack API and the token is used before it expires.

Expected Outcome

A SAML 2.0 assertion from a trusted IdP is exchanged for an OAuth2 access token with the SAML2 bearer grant, and the token is used to call an API through the Gateway.

CC BY 4.0